

Complete RAG (Retrieval- Augmented Generation) Pipeline (2026)

What is RAG?

Retrieval-Augmented Generation (RAG) is an AI architecture that combines the reasoning ability of Large Language Models (LLMs) with external knowledge retrieval. Instead of relying only on the knowledge stored inside the model during training, RAG searches relevant documents from a knowledge base and provides them as context before generating an answer.

This approach allows AI systems to answer questions using up-to-date, domain-specific, and private information while significantly reducing hallucinations.

Complete RAG

Pipeline

Data Sources



Data Collection



Document Cleaning



Chunking



Metadata Creation



Embedding Generation



Vector Database

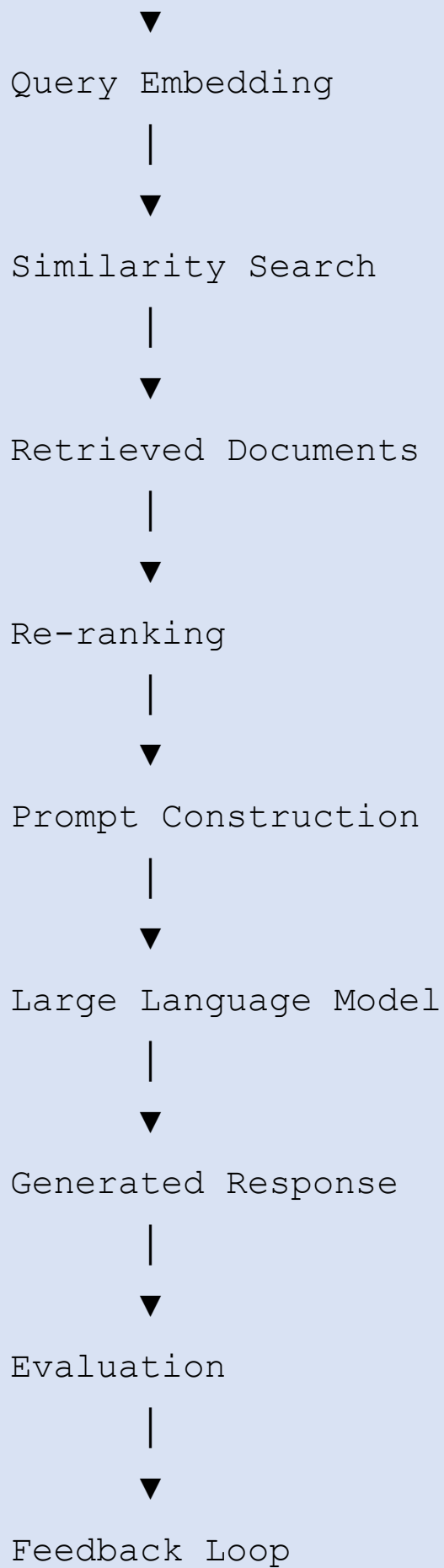


User Query



Query Processing





Step 1: Data Collection

Everything starts with collecting documents.

Possible sources include:

- PDFs
- Word files
- Excel files
- Websites
- APIs
- SQL Databases
- NoSQL Databases
- Cloud Storage
- Emails
- Markdown files
- Notion

- Confluence
- SharePoint
- GitHub repositories
- Customer support tickets

Goal:

Create a centralized knowledge base.

Output:

Raw documents.

Step 2: Document Parsing

Raw documents cannot be directly indexed.

The parser extracts meaningful content.

Tasks include:

- Extract text
- Remove images (optional)
- Extract tables
- Read headers
- Extract footers
- Preserve page numbers
- Detect sections

Popular Libraries

- PyMuPDF
- pdfplumber
- Apache Tika
- Unstructured
- BeautifulSoup
- Selenium

Output:

Clean text.

Step 3: Data Cleaning

Before embedding, clean the data.

Cleaning includes:

- Remove duplicate spaces
- Remove HTML
- Remove unnecessary symbols
- Normalize Unicode
- Fix encoding issues
- Remove advertisements
- Remove repeated headers
- Remove page numbers
- Standardize formatting

Goal:

Improve embedding quality.

Step 4: Text

Chunking

LLMs cannot process extremely long documents.

Split documents into smaller chunks.

Types of Chunking

Fixed Chunking

Example

500 words

1000 words

Simple but may cut important information.

Sentence Chunking

Split using sentences.

Advantages

- Better context
 - Natural boundaries
-

Paragraph Chunking

Entire paragraph becomes one chunk.

Useful for articles.

Semantic Chunking

Uses AI to split according to meaning.

Most accurate.

Recursive Chunking

Splits using

- Heading
- Paragraph
- Sentence
- Word

Most widely used approach.

Sliding Window Chunking

Each chunk overlaps with previous chunk.

Example

Chunk 1

Sentence 1–10

Chunk 2

Sentence 8–18

Benefits

Maintains context continuity.

Step 5: Metadata Generation

Each chunk should contain metadata.

Example

Document Name

Page Number

Section

Author

Date

Source

Language

Department

URL

Tags

Category

Chunk ID

Metadata improves filtering during retrieval.

Step 6: Embedding Generation

Text must be converted into vectors.

Embedding models understand semantic meaning.

Example

"The cat is sleeping"

↓

[0.24, 0.51, 0.91, ...]

Popular Embedding Models

OpenAI

BGE

E5

Jina Embeddings

Nomic Embed

Sentence Transformers

Instructor XL

Cohere Embed

Voyage AI

MixedBread AI

Step 7: Vector Database

Embeddings are stored in a vector database.

Popular Vector Databases

- Pinecone
- Weaviate
- Milvus
- Qdrant
- ChromaDB
- FAISS
- Elasticsearch
- Vespa
- LanceDB
- pgvector

Stored Information

- Embedding
- Chunk
- Metadata
- Document ID

Step 8: Indexing

Vector databases create indexes for fast searching.

Popular algorithms

HNSW

IVF

Flat Index

PQ

DiskANN

Benefits

- Faster search
- Lower latency
- Better scalability

Step 9: User Query

User asks a question.

Example

How does health insurance work?

This becomes the retrieval query.

Step 10: Query Processing

Before retrieval, preprocess the query.

Tasks

Lowercase

Remove noise

Spell correction

Expand abbreviations

Language detection

Query rewriting

Keyword extraction

Intent detection

Entity recognition

Output

Optimized query.

Step 11: Query Embedding

The query is converted into an embedding using the same embedding model.

Example

User Query

↓

Embedding

↓

Vector

Now the vector database can compare similarity.

Step 12: Similarity Search

Find the closest vectors.

Distance Metrics

Cosine Similarity

Dot Product

Euclidean Distance

Maximum Inner Product

Output

Top-K relevant chunks.

Example

Top 5

Top 10

Top 20

Step 13: Metadata Filtering

Sometimes retrieval needs filtering.

Examples

Department = HR

Language = English

Date > 2025

Country = India

Product = Laptop

Metadata filtering increases precision.

Step 14: Hybrid Search

Modern RAG systems combine two retrieval methods.

Dense Search

Uses embeddings.

Finds semantic similarity.

Sparse Search

Uses keywords.

BM25

TF-IDF

Inverted Index

Hybrid Search combines both.

Benefits

Higher recall

Better precision

Fewer missed documents

Step 15: Re-ranking

Initial retrieval is not always perfect.

A re-ranker scores retrieved chunks again.

Instead of embedding similarity, it understands relationships between query and documents.

Popular Re-rankers

Cross Encoder

BGE Reranker

Cohere Rerank

Jina Reranker

NVIDIA Reranker

Result

Most relevant chunks appear first.

Step 16: Context Compression

Retrieved documents may exceed token limits.

Compression removes unnecessary text.

Methods

Summarization

Sentence selection

Redundancy removal

Context pruning

Result

Higher quality prompt.

Step 17: Prompt Engineering

Construct the final prompt.

Example

System Prompt

+

User Question

+

Retrieved Context

+

Instructions

↓

LLM

Good prompts reduce hallucinations.

Prompt usually includes

Role

Instructions

Context

Examples

Formatting

Restrictions

Output format

Step 18: Large Language Model

The LLM receives

User Question

Retrieved Context

Instructions

Examples

The LLM reasons over retrieved documents.

Popular Models

GPT

Claude

Gemini

Llama

Mistral

DeepSeek

Qwen

Command R

Phi

Yi

Step 19: Response Generation

The LLM generates

Answer

Summary

Explanation

Code

Table

Bullet Points

Citations

JSON

Markdown

Structured Output

Step 20: Source Attribution

Good RAG systems provide references.

Example

Answer

Source PDF

Page 18

Section 4

Confidence

Benefits

Transparency

Trust

Fact verification

Step 21: Guardrails

Production RAG requires safety.

Guardrails include

Prompt injection detection

PII removal

Sensitive data filtering

Toxicity detection

Content moderation

Policy enforcement

Output validation

Step 22: Post-processing

Before returning the answer

Grammar correction

Formatting

Markdown conversion

Citation formatting

JSON validation

Language translation

Output cleanup

Step 23: Response Delivery

The final response reaches

Web application

Chatbot

Slack

WhatsApp

Teams

Mobile App

Voice Assistant

API

Browser extension

Step 24: Logging

Store

Question

Retrieved chunks

Prompt

Model output

Latency

Token usage

Errors

User feedback

Logs help debugging.

Step 25: Evaluation

Evaluate RAG quality.

Metrics

Context Precision

Context Recall

Faithfulness

Answer Relevancy

Groundedness

Hallucination Rate

Latency

Token Cost

Retrieval Accuracy

Citation Accuracy

Step 26: User Feedback

Collect feedback.

Examples

 Helpful

 Not Helpful

Star Rating

Comments

Feedback improves retrieval.

Step 27: Continuous Improvement

Improve the pipeline continuously.

Activities

Update documents

Rebuild embeddings

Optimize chunk size

Improve prompts

Train better rerankers

Monitor latency

Reduce hallucinations

Improve caching

Advanced Components (2026)

Multi-Vector Retrieval

One document has multiple embeddings.

Examples

Title embedding

Body embedding

Summary embedding

Table embedding

Image embedding

Improves retrieval quality.

Parent-Child Retrieval

Store

Small chunks

Large parent documents

Retrieve

Small chunk

Return

Large parent context

Provides detailed answers.

Multi-Query Retrieval

Generate multiple search queries.

Example

Original

"What is cloud computing?"

Generated

Explain cloud computing

Cloud definition

Cloud architecture

Cloud basics

Retrieve using all queries.

Self-RAG

The LLM evaluates whether retrieved information is sufficient.

If not

Retrieve again

Repeat until confidence is high.

Adaptive RAG

Different retrieval strategy based on query complexity.

Simple query

One retrieval

Complex query

Multiple retrieval rounds

Graph RAG

Uses knowledge graphs.

Retrieves entities and relationships.

Excellent for enterprise knowledge.

Agentic RAG

AI agents perform

Planning

Tool calling

Multiple searches

Reasoning

Memory

Verification

Final answer generation

Ideal for complex workflows.

Multimodal RAG

Retrieves

Images

Charts

Audio

Video

Tables

PDFs

Slides

CAD files

Medical images

Satellite images

The LLM reasons across multiple data types.

Long-Context RAG

Uses models with very large context windows.

Advantages

Fewer chunks

Reduced retrieval complexity

Better document understanding

Cache Layer

Improve speed using caches.

Types

Embedding Cache

Query Cache

Prompt Cache

Response Cache

Semantic Cache

Benefits

Lower cost

Lower latency

Higher throughput

End-to-End RAG

Workflow

Raw Data



Data Collection



Document Parsing



Data Cleaning

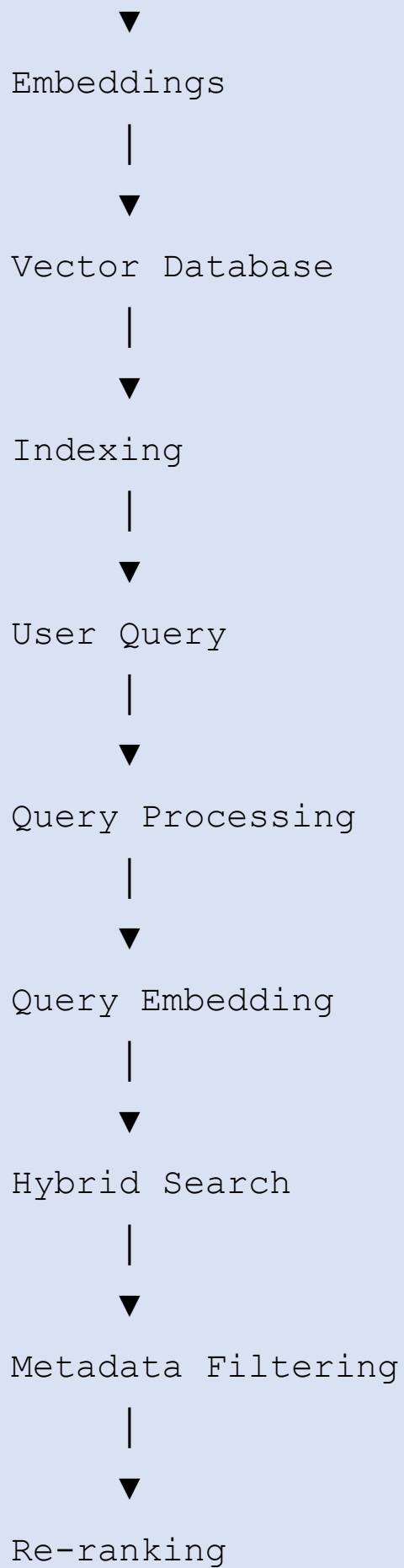


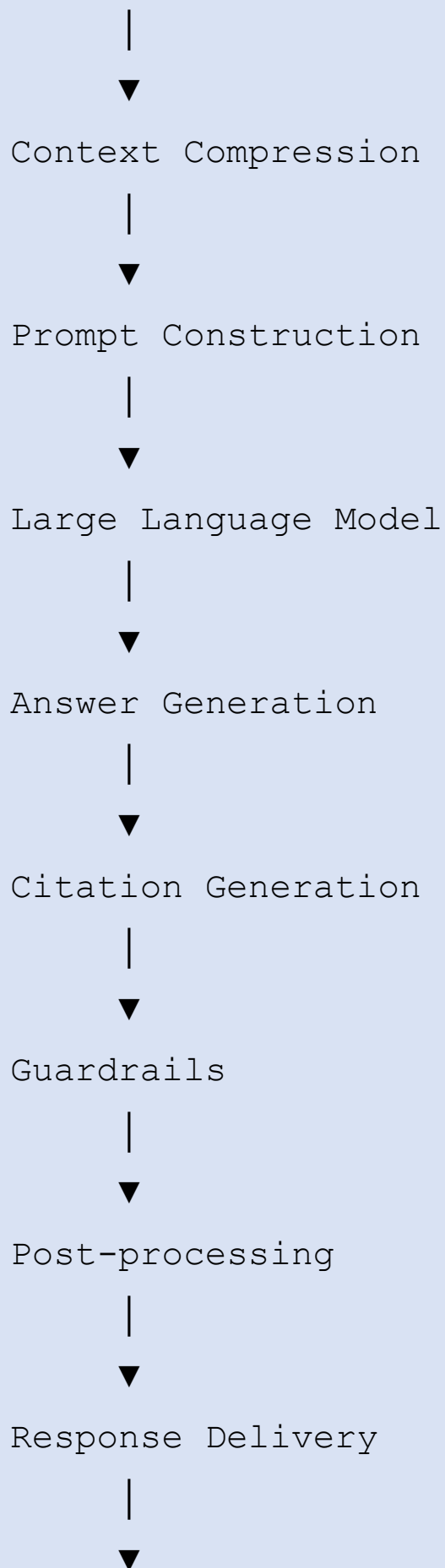
Chunking



Metadata







Logging



Evaluation



Feedback



Continuous Improvement